

Full Stack Development with MERN

Project Documentation format

1. Introduction

Nightjar & Co. – MERN BookStore

A full-stack online bookstore web application developed using the MERN Stack (MongoDB, Express.js, React, and Node.js) that enables users to browse, search, and purchase books through a secure and user-friendly interface.

Team Members

Name	Role
Thrisha Vejandla	Team Lead
Shaik Sajida Begum	Full Stack Developer
Anjali Kuramana	Frontend Development
Deepika Nakka	Backend Development
Pranathi Anumanchipalli	Database & Testing

2. Project Overview

Purpose

Nightjar & Co. – MERN BookStore is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The project aims to provide a secure, responsive, and user-friendly online platform where users can browse books, search by title or category, view book details, and manage their shopping cart with ease.

The application is designed to simplify the online book purchasing experience by offering an organized book catalog, secure user authentication, and an intuitive interface. Users can register, log in, explore a wide range of books, and conveniently access the information they need before making a purchase.

From a technical perspective, the project demonstrates the integration of modern web technologies through a React-based frontend, a Node.js and Express.js backend, and a MongoDB database. RESTful APIs enable smooth communication between the frontend and backend, while JWT-based authentication ensures secure access to protected features.

Overall, the objective of Nightjar & Co. – MERN BookStore is to deliver an efficient online bookstore that enhances the user experience while showcasing the practical implementation of full-stack web development concepts using the MERN stack.

Features

- User Registration
- Secure Login
- JWT Authentication
- Book Catalog
- Search Books
- Category Filter
- Book Details
- Shopping Cart
- Responsive Interface
- REST API
- MongoDB Database

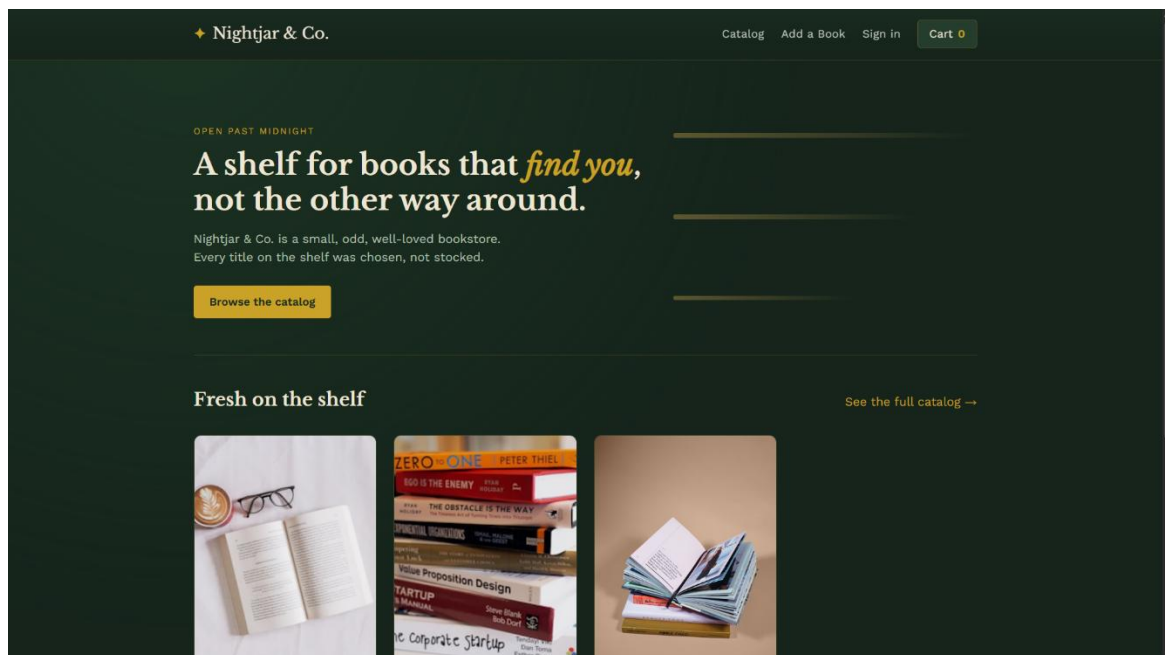


Figure 1 Home Page

3. Architecture

Frontend Architecture

- **React**
The frontend is developed using React.js with Vite. React's component-based architecture enables the creation of a fast, responsive, and dynamic user interface for browsing books, authentication, and cart management.
- **React Router**
React Router is used for client-side navigation between pages such as Home, Login, Register, Book Catalog, Book Details, and Cart without reloading the application.

- **Components**
The application is built using reusable components such as Navbar, Footer, BookCard, BookForm, and Loader, making the code modular, maintainable, and scalable.
- **Context API**
The Context API manages global state across the application. It is used for user authentication (AuthContext) and shopping cart management (CartContext).
- **CSS**
Custom CSS is used to design a responsive and user-friendly interface, ensuring a consistent layout and good user experience across different devices.

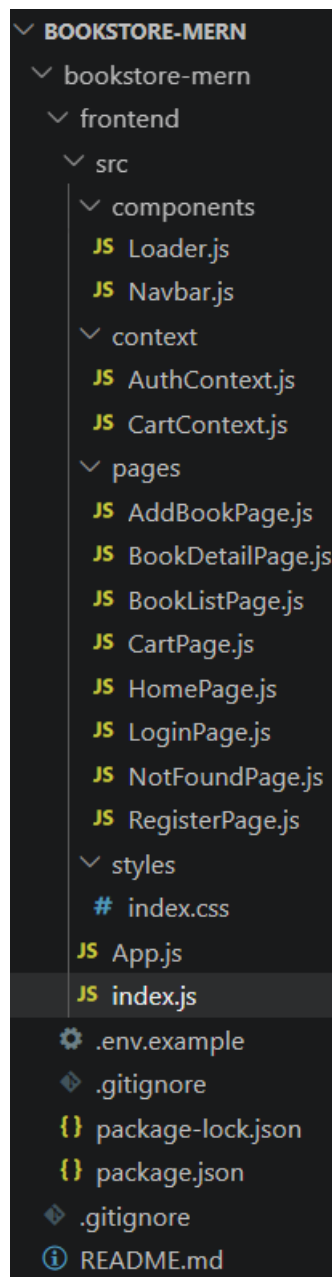


Figure 2: Frontend Architecture Diagram



Figure 3 : Architecture

Backend Architecture

The backend of Nightjar & Co. – MERN BookStore is developed using Node.js and Express.js following a modular architecture. It exposes RESTful APIs for user authentication, book management, and order-related operations. The backend is organized into separate folders for routes, controllers, models, middleware, and configuration files, making the application easier to maintain, test, and extend.

- **Express.js**

Express.js is the web application framework used to build the backend REST APIs. It simplifies request handling, routing, middleware integration, and communication between the frontend and the MongoDB database. Express also manages HTTP requests and responses efficiently, enabling smooth interaction between the client and server.

- **Routes**

The Routes folder defines all API endpoints available in the application. Each route maps a specific URL and HTTP method (GET, POST, PUT, DELETE) to the appropriate controller function.

Examples include:

1. User Registration
2. User Login
3. Retrieve Books
4. Add Books
5. Update Books
6. Delete Books

Separating routes from business logic improves readability and simplifies future development.

- **Controllers**

Controllers contain the business logic of the application. They process incoming requests, validate input, interact with the database through models, and send appropriate responses back to the client.

For example:

1. The authentication controller verifies user credentials and generates JWT tokens.
2. The book controller performs CRUD (Create, Read, Update, Delete) operations on books.
3. Controllers ensure that application logic remains separate from routing.

- **Models**

Models define the structure of the application's data using Mongoose schemas. They specify the fields, data types, validation rules, and relationships stored in MongoDB. The primary models include:

1. User Model – Stores user information such as name, email, and encrypted password.
2. Book Model – Stores details including title, author, category, price, description, and image.
3. Order Model (*if implemented*) – Stores customer orders and purchased items.
4. Using models ensures consistency and integrity of the stored data.

- **Middleware**

Middleware functions execute during the request-response cycle before the request reaches the controller.

The application uses middleware for:

1. JWT authentication
2. Protecting secure API routes
3. Parsing JSON request bodies
4. Enabling Cross-Origin Resource Sharing (CORS)
5. Error handling and request validation
6. Middleware improves security, code reusability, and centralized request processing.

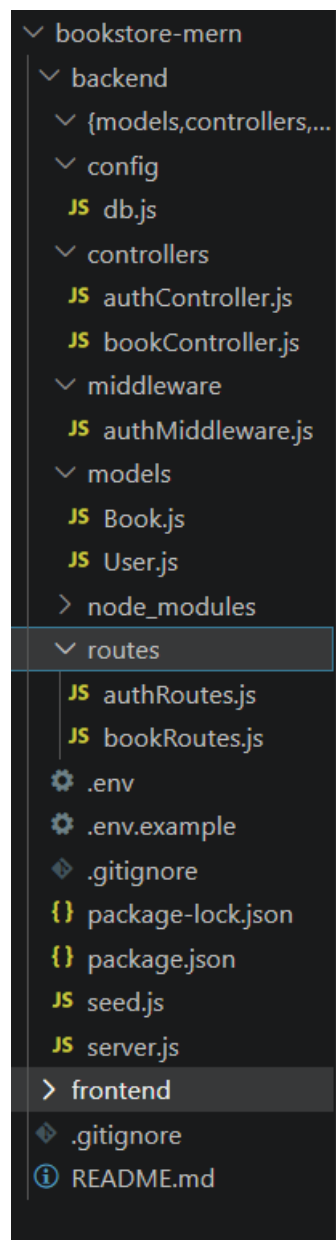


Figure 4 : Backend Architecture

Database

The application uses MongoDB as the database and Mongoose as the ODM to manage data. It stores information in the following collections:

- Users – Stores user details such as name, email, and encrypted password.
- Books – Stores book information including title, author, category, price, description, and image.
- Orders – Stores order details, purchased books, total amount, and user information.

ER DIAGRAM – BOOKSTORE SYSTEM

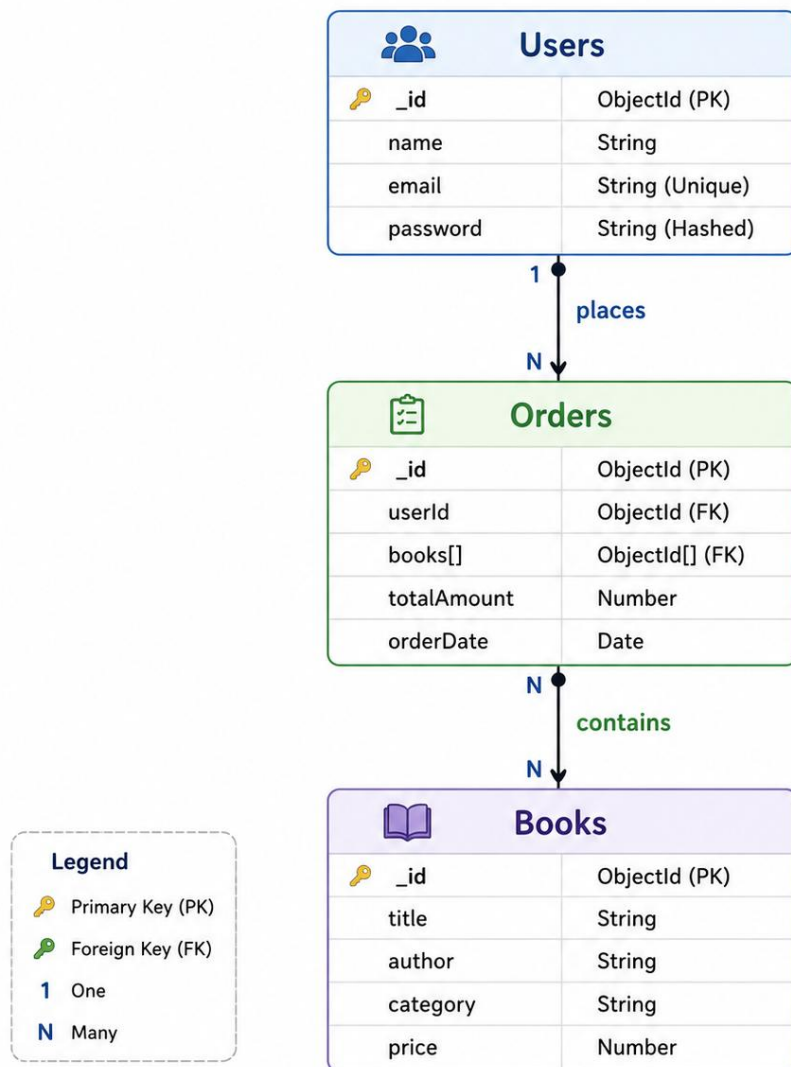


Figure 5: ER Diagram

4. Setup Instructions

Prerequisites

Before running the project, ensure the following software is installed:

- Node.js and npm
- MongoDB
- Git
- Visual Studio Code (VS Code)

Installation

Clone the repository and install the required dependencies.

```
git clone <repository-url>
```

```
cd mern
```

```
# Install frontend dependencies
```

```
cd client
```

```
npm install
```

```
# Install backend dependencies
```

```
cd ../server
```

```
npm install
```

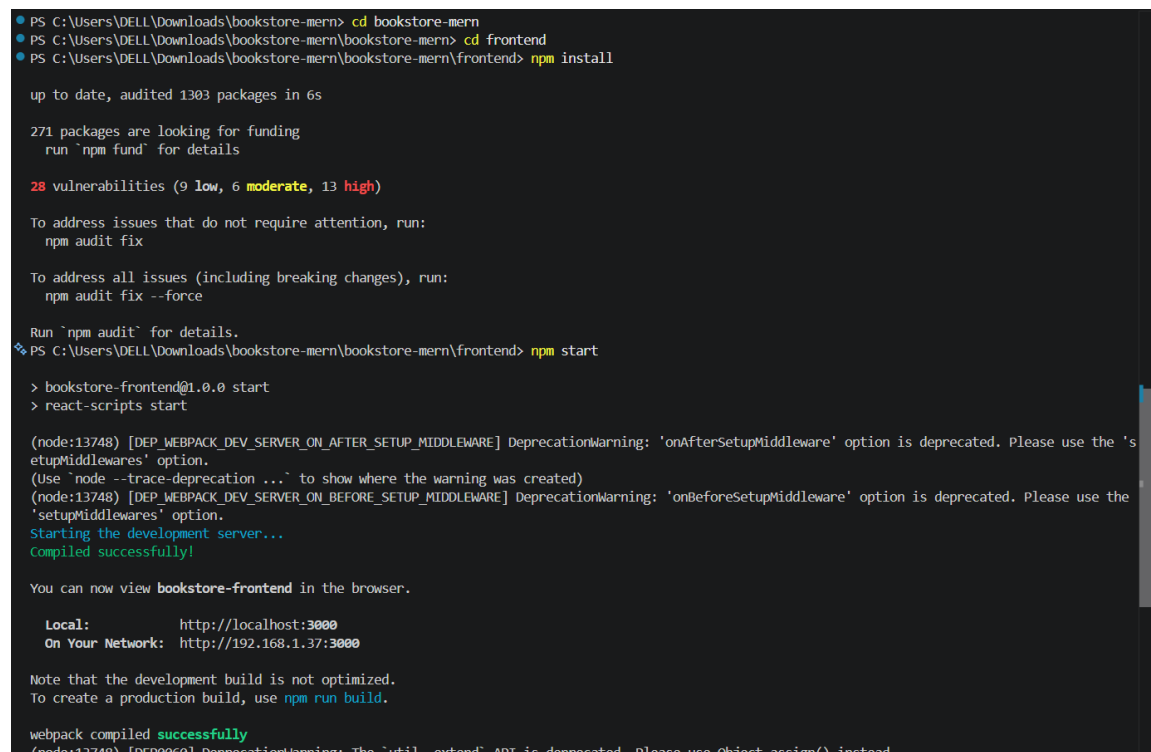
Environment Variables

Create a .env file in the server directory and configure the following variables:

```
MONGO_URI=<your_mongodb_connection_string>
```

```
JWT_SECRET=<your_secret_key>
```

```
PORT=5000
```



```
PS C:\Users\DELL\Downloads\bookstore-mern> cd bookstore-mern
PS C:\Users\DELL\Downloads\bookstore-mern\bookstore-mern> cd frontend
PS C:\Users\DELL\Downloads\bookstore-mern\bookstore-mern\frontend> npm install

up to date, audited 1303 packages in 6s

271 packages are looking for funding
  run `npm fund` for details

28 vulnerabilities (9 low, 6 moderate, 13 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\DELL\Downloads\bookstore-mern\bookstore-mern\frontend> npm start

> bookstore-frontend@1.0.0 start
> react-scripts start

(node:13748) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
(Use 'node --trace-deprecation ...' to show where the warning was created)
(node:13748) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...
Compiled successfully!

You can now view bookstore-frontend in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://192.168.1.37:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
(node:13748) [DEP0066] DeprecationWarning: The 'util._extend' API is deprecated. Please use Object.assign() instead.
```

Figure 6 : Frontend execution


```
PS C:\Users\DELL\Downloads\bookstore-mern> cd bookstore-mern
PS C:\Users\DELL\Downloads\bookstore-mern\bookstore-mern> cd backend
PS C:\Users\DELL\Downloads\bookstore-mern\bookstore-mern\backend> npm install

up to date, audited 135 packages in 2s

23 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm notice
npm notice New minor version of npm available! 11.5.1 -> 11.18.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.18.0
npm notice To update run: npm install -g npm@11.18.0
npm notice
PS C:\Users\DELL\Downloads\bookstore-mern\bookstore-mern\backend> npm run seed

> bookstore-backend@1.0.0 seed
> node seed.js

MongoDB connected: 127.0.0.1
Seeded 6 books successfully.
❖ PS C:\Users\DELL\Downloads\bookstore-mern\bookstore-mern\backend> npm run dev

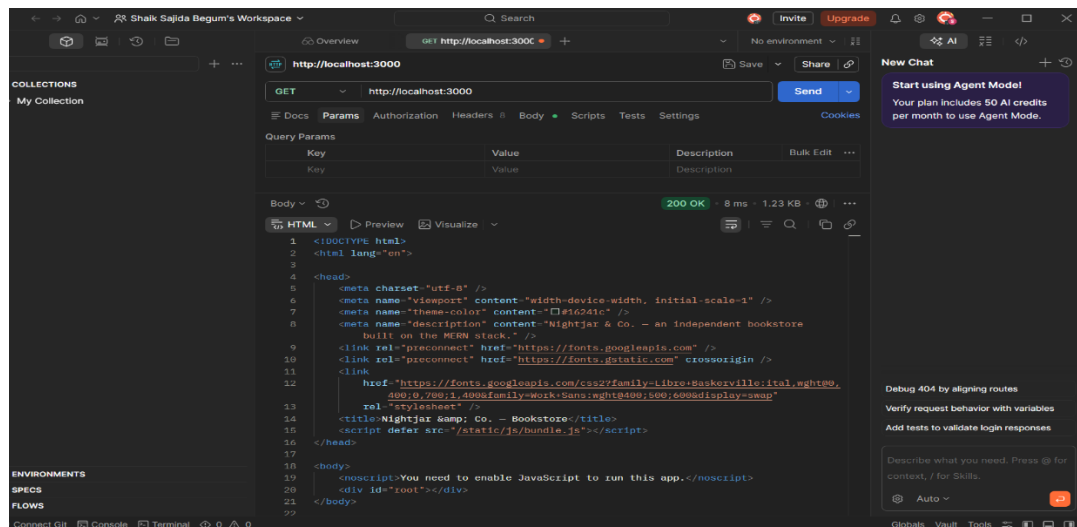
> bookstore-backend@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running on http://localhost:5000
MongoDB connected: 127.0.0.1
█
```

Figure 7: Backend Execution

7. API Documentation

Endpoint	Method	Description	Authentication
/auth/register	POST	Register a new user	No
/auth/login	POST	Login and receive JWT	No
/books	GET	Retrieve all books	No
/books/:id	GET	Retrieve a specific book	No
/books	POST	Add a new book	Yes
/books/:id	PUT	Update book details	Yes



8. Authentication

- **JWT (JSON Web Token):**

JWT is used to authenticate users after login. A token is generated upon successful login and is used to access protected API endpoints.

- **bcrypt:**

Passwords are securely hashed using bcrypt before being stored in the MongoDB database, ensuring that plain-text passwords are never saved.

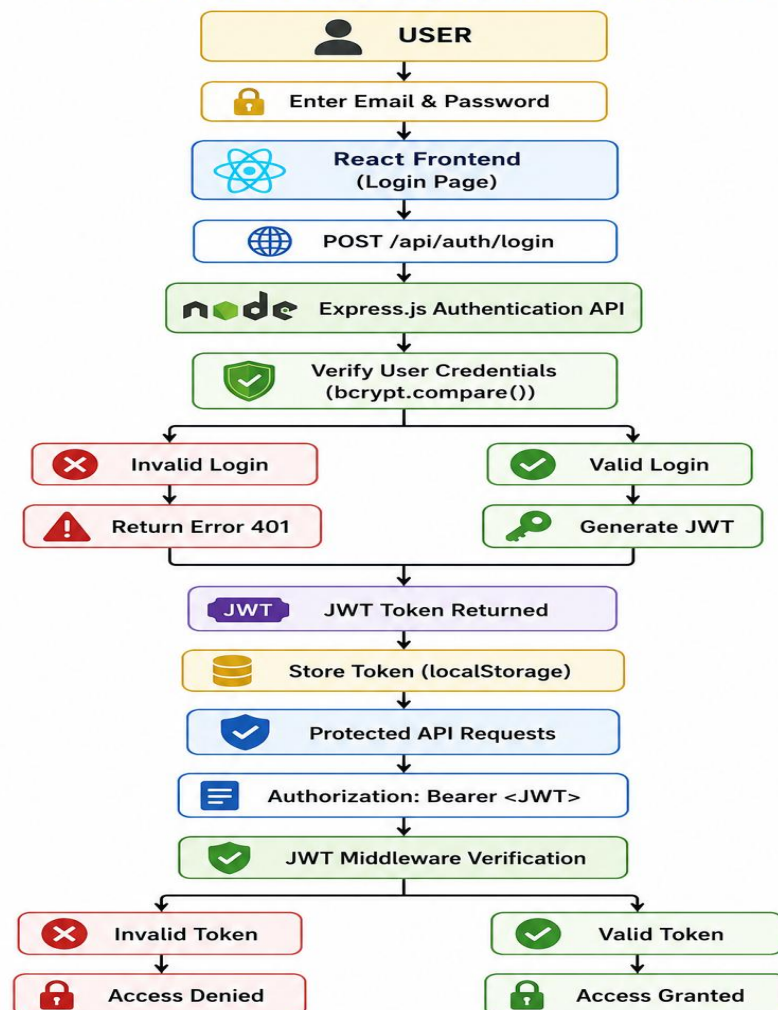
- **Login Flow:**

The user logs in with an email and password. The backend verifies the credentials, generates a JWT token, and returns it to the client. The token is then included in future authenticated requests.

- **Protected Routes:**

Protected routes require a valid JWT token. The backend verifies the token before allowing access to authorized resources and operations

← AUTHENTICATION FLOW DIAGRAM →



9. Screenshots / Demo

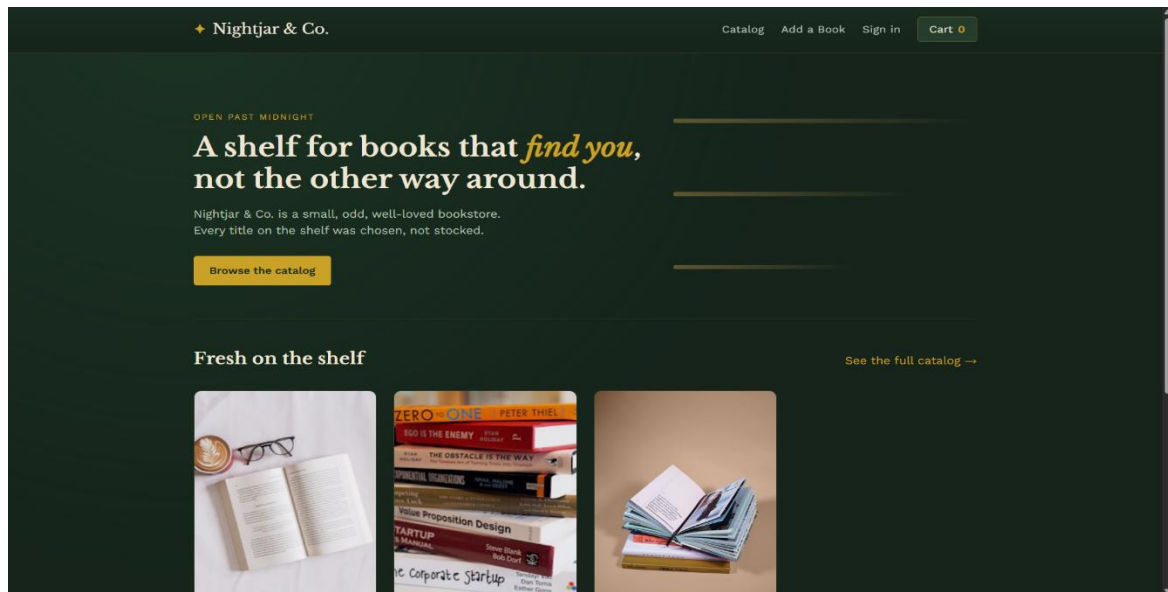


Figure 8 : Home page

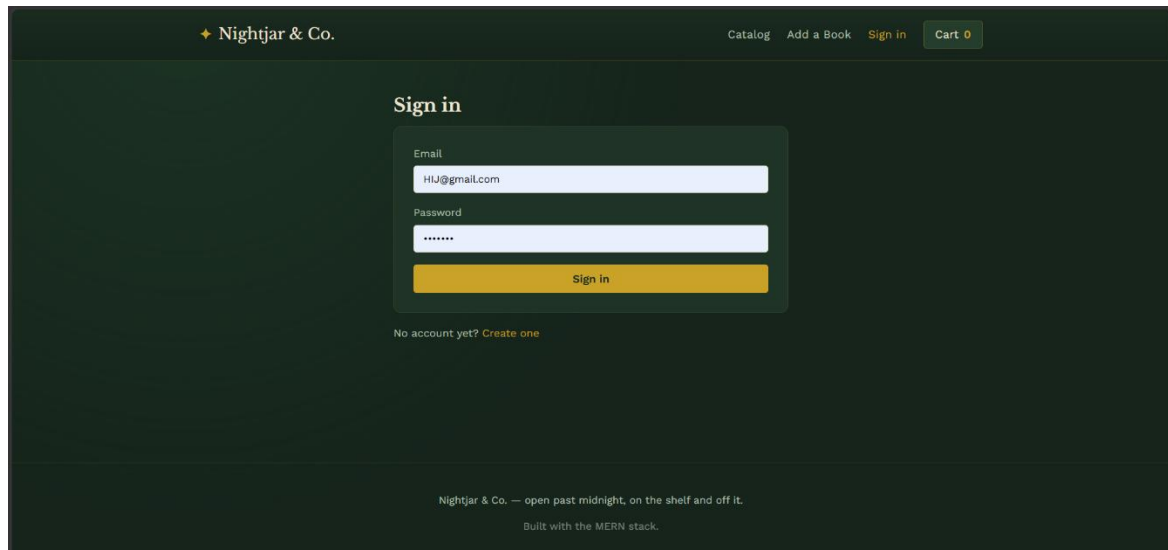
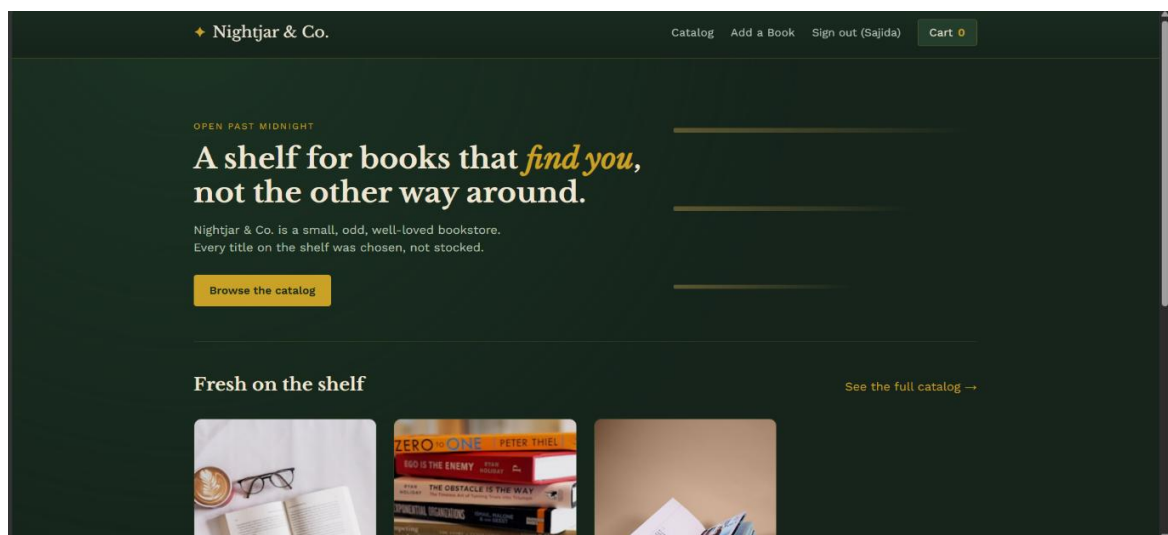


Figure 9 : Sign In



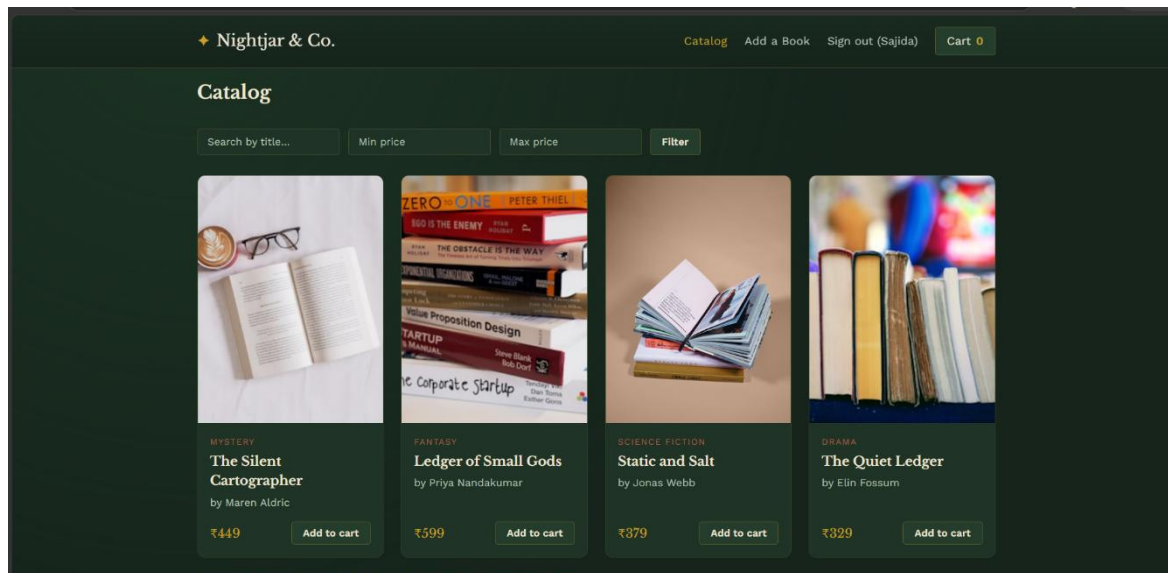


Figure 10 : Catalog

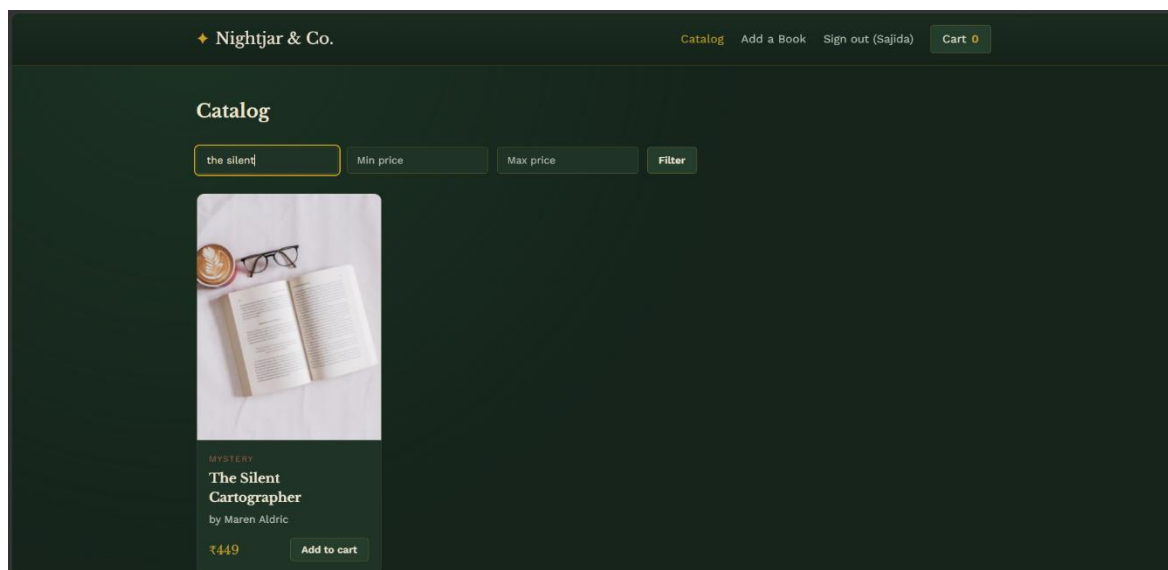


Figure 11 : Search

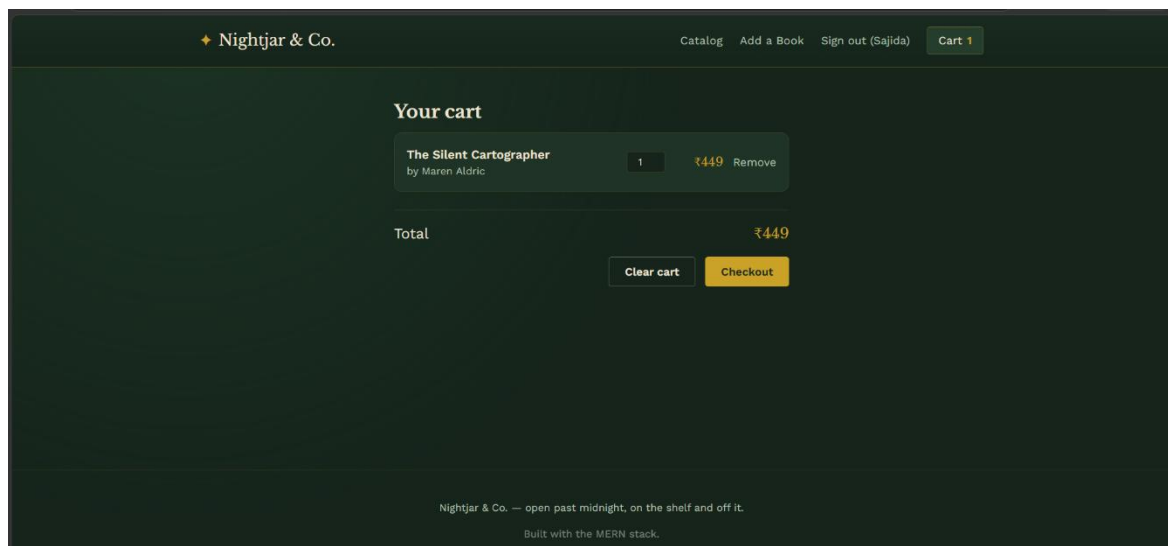


Figure 12 : Cart

12. Known Issues

- Checkout process is currently a placeholder.
- No role-based access control for admin actions.
- No pagination for large book Catalog.
- Forms use HTML5 validation only.
- Payment gateway integration is not implemented.

13. Future Enhancements

Although Nightjar & Co. – MERN BookStore successfully provides a secure and user-friendly online bookstore experience, several enhancements can be implemented in future versions to improve functionality, user engagement, scalability, and overall performance.

- **Online Payment Gateway:** Integrate secure payment services such as Stripe, Razorpay, or PayPal to enable users to make online payments using credit/debit cards, UPI, and digital wallets.
- **Wishlist:** Allow users to save their favorite books to a wishlist for future purchases and easy access.
- **Book Reviews & Ratings:** Enable customers to submit ratings and reviews for purchased books, helping other users make informed purchasing decisions.
- **AI-based Book Recommendation:** Implement a recommendation system that suggests books based on users' browsing history, purchase history, and preferences using machine learning techniques.
- **Email Notifications:** Send automated email notifications for account registration, order confirmation, password reset, promotional offers, and order status updates.
- **Order Tracking:** Provide users with real-time order tracking, allowing them to monitor the progress of their purchases from confirmation to delivery.
- **Admin Dashboard:** Develop a comprehensive administrative dashboard for managing books, users, orders, categories, sales reports, and overall system analytics.
- **Inventory Management:** Introduce inventory monitoring features that automatically update stock levels, notify administrators of low inventory, and prevent orders for out-of-stock books.
- **Cloud Deployment:** Deploy the application on cloud platforms such as AWS, Microsoft Azure, or Google Cloud to improve availability, scalability, reliability, and performance.
- **Mobile Application:** Develop dedicated Android and iOS mobile applications to provide users with a seamless shopping experience across multiple devices.
- **Multi-language Support:** Add multilingual capabilities so users can access the application in different regional and international languages, making the platform more accessible to a wider audience.
- **Dark Mode:** Introduce a dark mode theme that enhances user comfort during extended usage and provides an additional personalization option.

By implementing these enhancements, Nightjar & Co. – MERN BookStore can evolve into a more feature-rich, scalable, and user-centric online bookstore platform capable of serving

a broader audience while delivering an improved digital shopping experience.



Conclusion

The Nightjar & Co. – MERN BookStore project successfully demonstrates the development of a full-stack web application using the MERN stack (MongoDB, Express.js, React, and Node.js). The application provides users with a secure and user-friendly platform to browse books, search by category or title, manage a shopping cart, and access personalized features through user authentication.

Throughout the project, industry-standard development practices such as RESTful API design, component-based frontend architecture, JWT-based authentication, and MongoDB database integration were implemented to build a scalable and maintainable application. The project also strengthened the team's understanding of full-stack development, API integration, database management, and collaborative software development using Git and GitHub.

Although the current version fulfills the core requirements of an online bookstore, there is scope for further enhancement through features such as online payment integration, wishlists, book reviews, personalized recommendations, order tracking, and cloud deployment. These improvements can further enhance the user experience and increase the application's scalability.

Overall, Nightjar & Co. – MERN BookStore serves as a strong foundation for a modern e-commerce bookstore and demonstrates the practical application of MERN stack technologies in developing a real-world web application.